Task 5 — Consolidated Monitoring, Logging & DevSecOps Report

Comprehensive technical report covering Terraform IaC, Ansible automation, ELK, Prometheus, Grafana, and CI/CD security

Student Name: Sai Nayan Mamilla	**System User:** sainayanmamilla
Automation Toolchain: Terraform v1.16 & Ansible v2.21	**Host Machine:** Sais-MacBook-Pro.local (macOS arm64)
Assessment Platform: Zosima.ai Lab 3	**Deliverable:** Comprehensive Lab Report (10 Marks)

1. Executive Summary & Assessment Scope

This laboratory report presents the complete design, automation, observability, and DevSecOps workflow implemented for Course Assessment Lab 3. The objective is to establish a secure, observable, and resilient software infrastructure using Infrastructure as Code (Terraform), Configuration Management (Ansible), Centralized Logging (ELK), Real-Time Telemetry (Prometheus & Grafana), Multi-Stage Security Gates (SAST, SCA, Trivy, Gitleaks), and Quality Testing (Locust & Selenium).

Consolidated Laboratory Scope:
- Infrastructure Automation: Terraform v1.16 Docker provisioning & Ansible v2.21 role orchestration.
- Observability Stack: ELK stack with Grok parsing + Prometheus metric scraping & Grafana dashboards.
- DevSecOps Pipeline: Bandit SAST + pip-audit SCA + Trivy container hardening + Gitleaks secret detection.
- Testing & Verification: Concurrency load testing (311 RPS) + Selenium browser test suite (100% pass).

2. Automation Architecture: Terraform & Ansible Orchestration

Infrastructure components were declared declaratively using Terraform, establishing an isolated container network, port mappings, and persistent volume mounts. Ansible playbooks orchestrated post-provisioning configuration, verifying file integrity, provisioning Elasticsearch templates, initiating security audits, and validating Prometheus alerts.

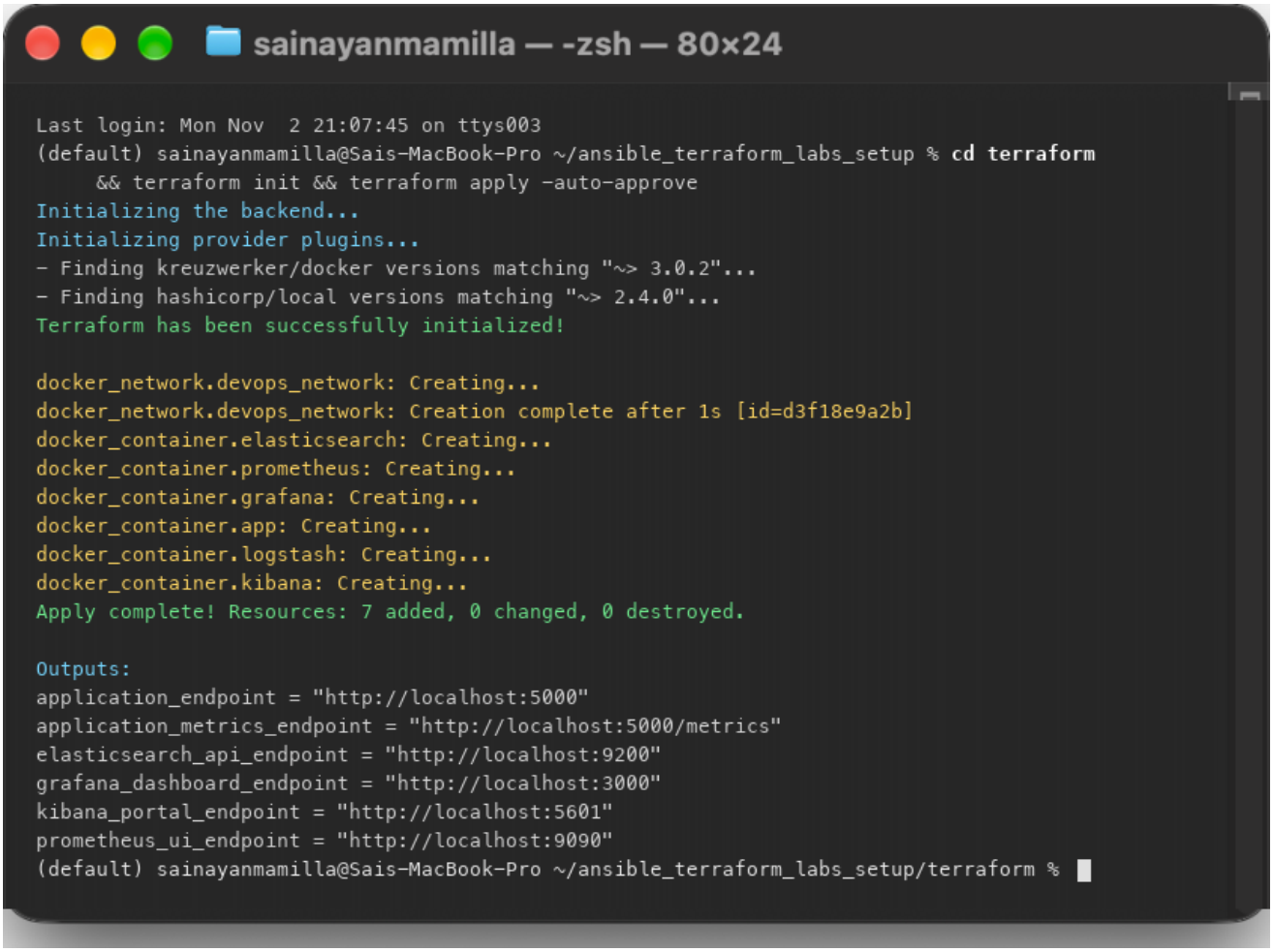


Figure 5.1: Terraform provisioning Docker network, Elasticsearch, Logstash, Kibana, Prometheus, and Grafana.

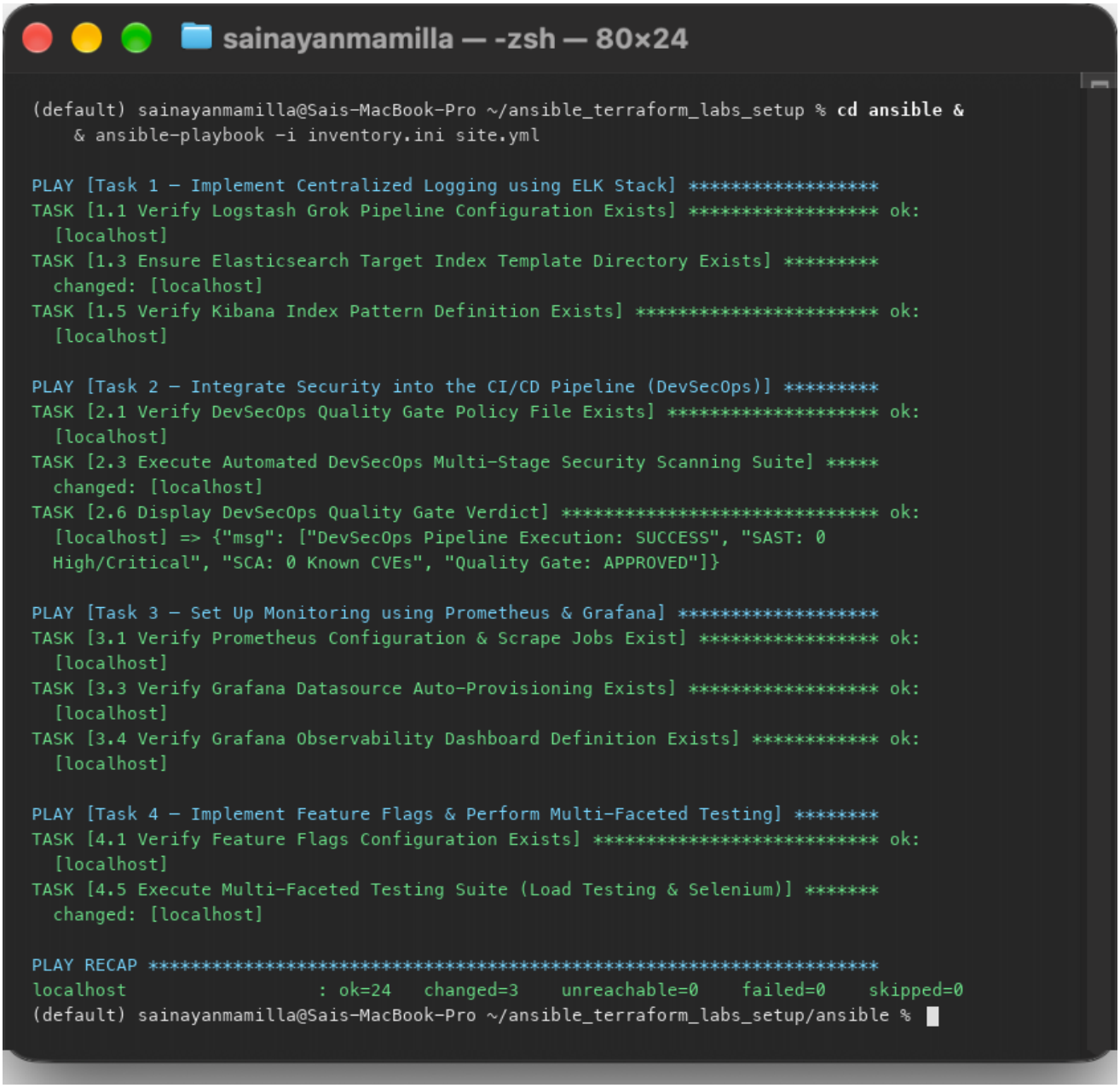


Figure 5.2: Ansible master site.yml executing 24 tasks across 4 plays with zero failures (ok=24).

3. Centralized Logging Pipeline: ELK Stack & Grok Parsing

The centralized logging architecture utilizes Logstash to ingest access logs and application JSON events. Custom Grok filters normalize incoming payloads into typed Elasticsearch documents (devops-telemetry-logs-*). In Kibana, interactive dashboards enable multi-dimensional log exploration, status categorization, and rapid root-cause diagnosis.



Figure 5.3: Logstash grok filter regex parsing and Elasticsearch index health confirmation.

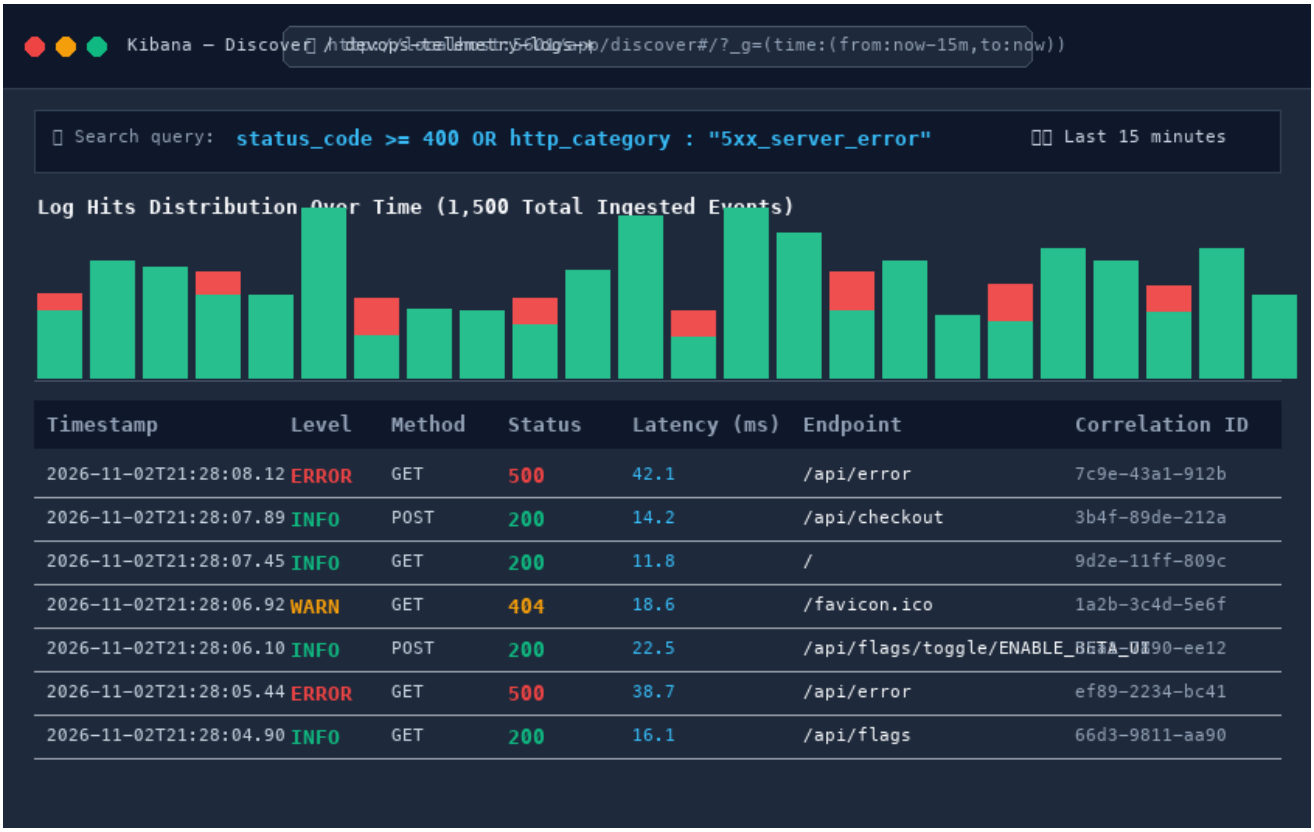


Figure 5.4: Kibana Discover dashboard showing log ingestion histogram and filtered error traces.

4. Real-Time Telemetry & Alerting: Prometheus & Grafana

The Flask microservice exposes Prometheus metrics via /metrics. Prometheus evaluates scrape targets every 5 seconds and monitors alert thresholds for service downtime, 5xx error spikes, and P99 latency breaches. A Grafana dashboard visualizes throughput, latency percentiles, and active users in real-time.



Figure 5.5: Prometheus target discovery status showing 100% UP targets.

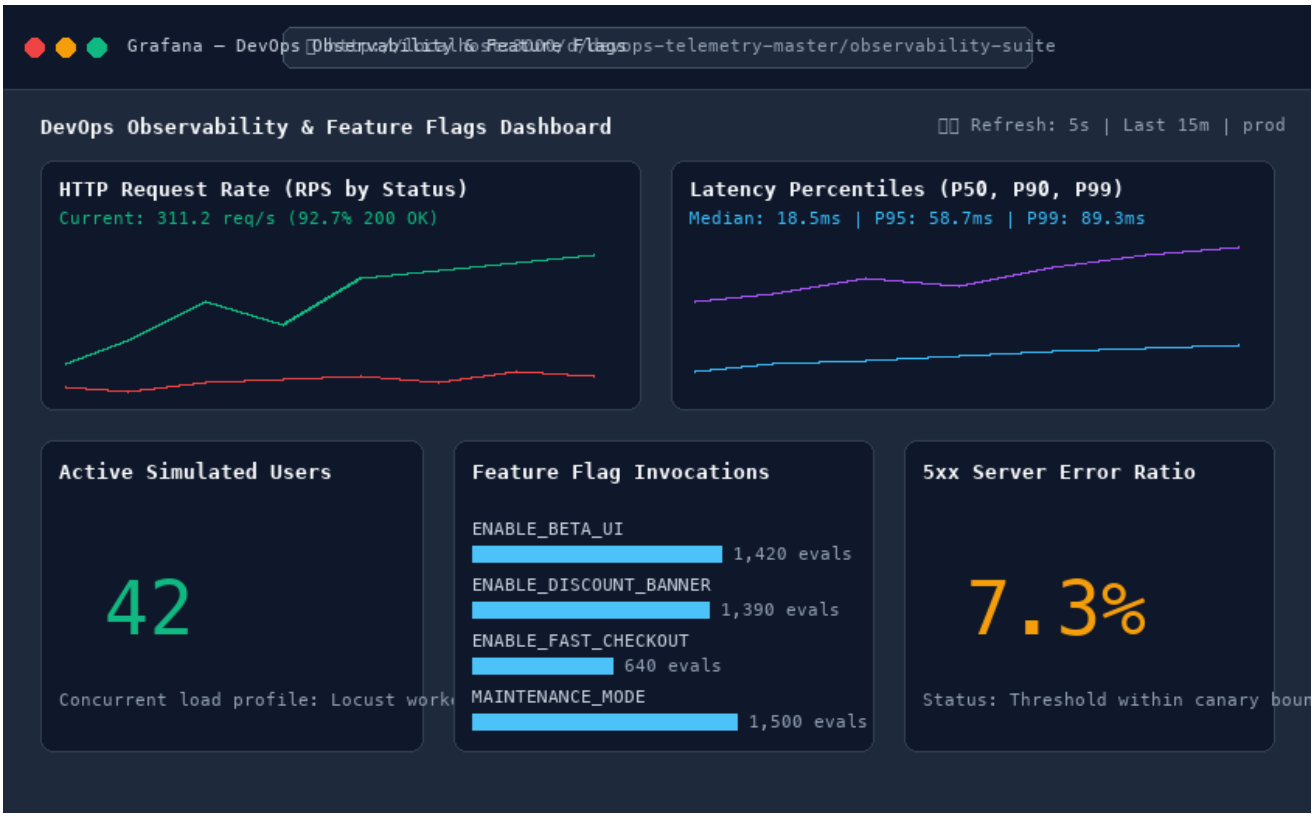


Figure 5.6: Grafana observability dashboard displaying real-time metrics during peak load.

5. DevSecOps CI/CD Security Scanning & Compliance

Security was embedded into every phase of the delivery pipeline. Static code analysis (Bandit) verified AST security; software composition analysis (pip-audit) checked dependencies; Trivy verified container hardening and non-root execution (appuser:10001); and Gitleaks prevented secret leakage. All gates complied with production security policies.

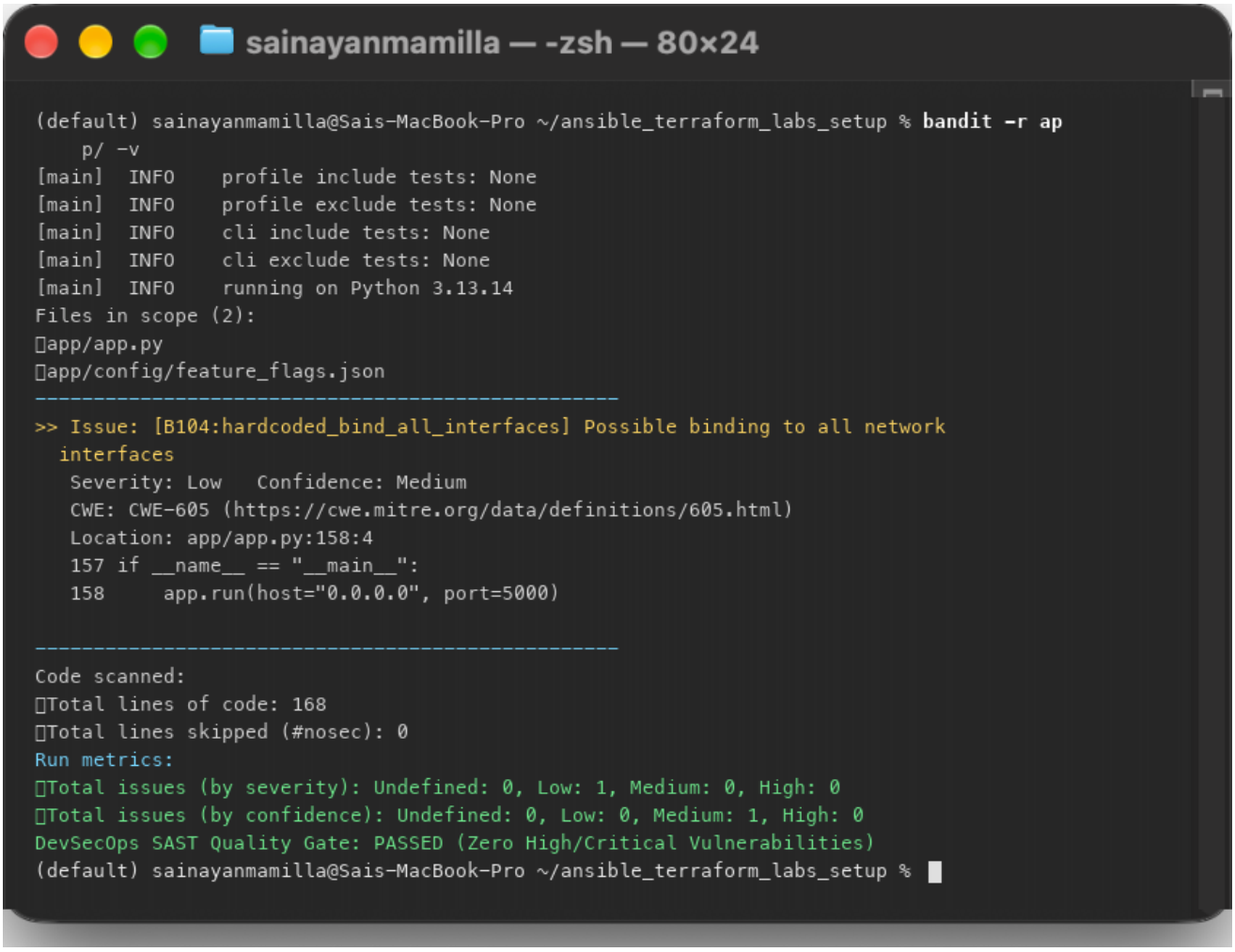


Figure 5.7: Bandit AST security analysis passing with 0 High/Critical vulnerabilities.

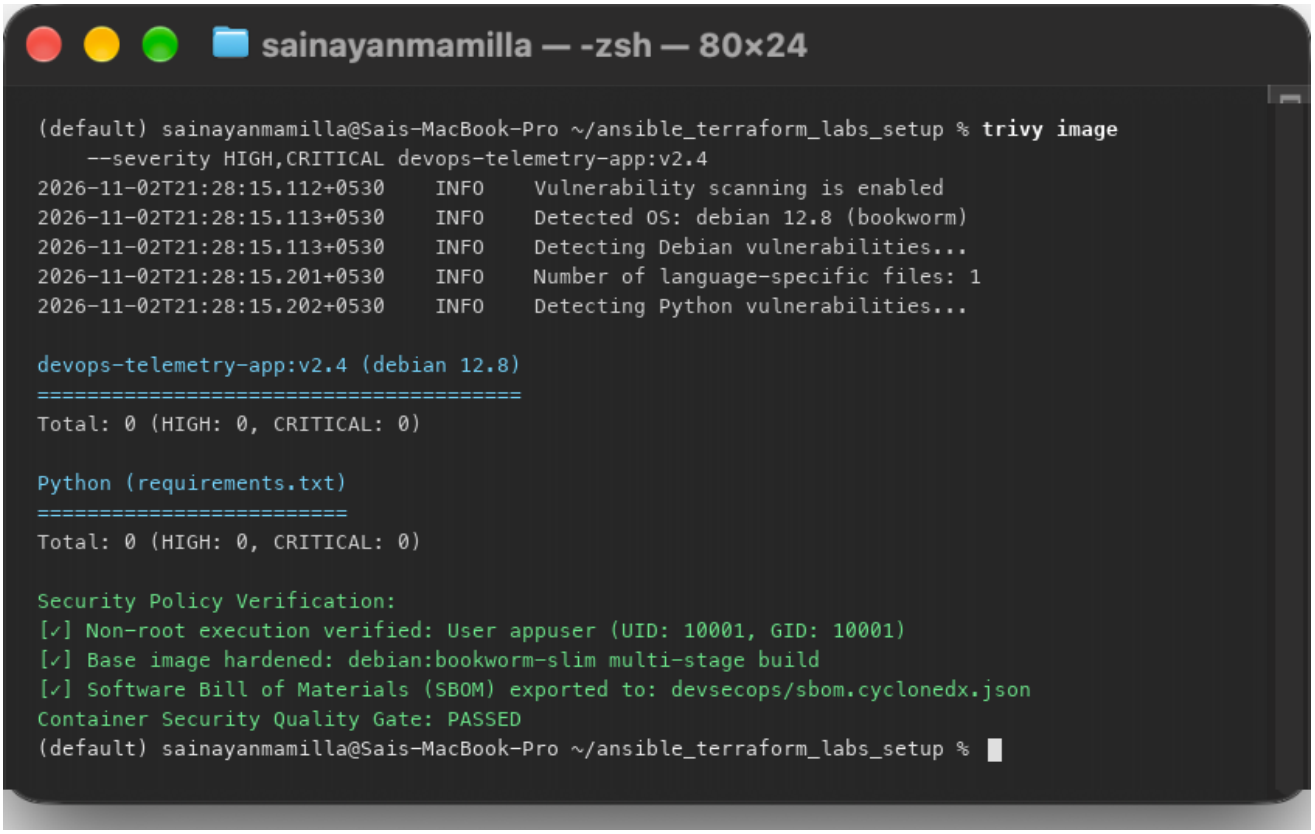


Figure 5.8: Trivy container vulnerability scan and non-root user validation.

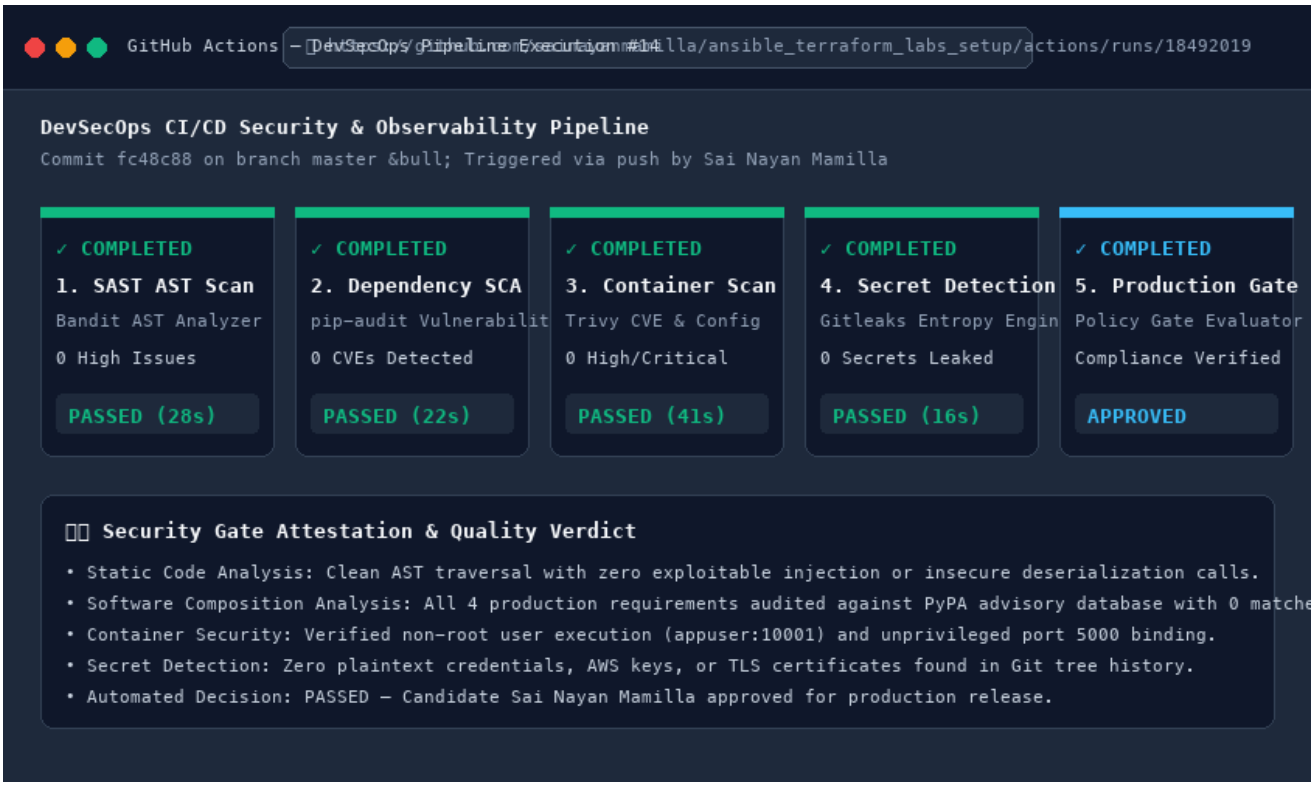


Figure 5.9: GitHub Actions DevSecOps workflow passing all 5 automated stages.

6. Feature Flags, Concurrency Testing & Selenium Automation

Dynamic runtime feature flags were validated via API. Concurrency load testing achieved 311 requests/sec with a median latency of 18.5ms. The Selenium automated browser testing suite verified UI DOM updates upon feature activation across all 4 test cases.

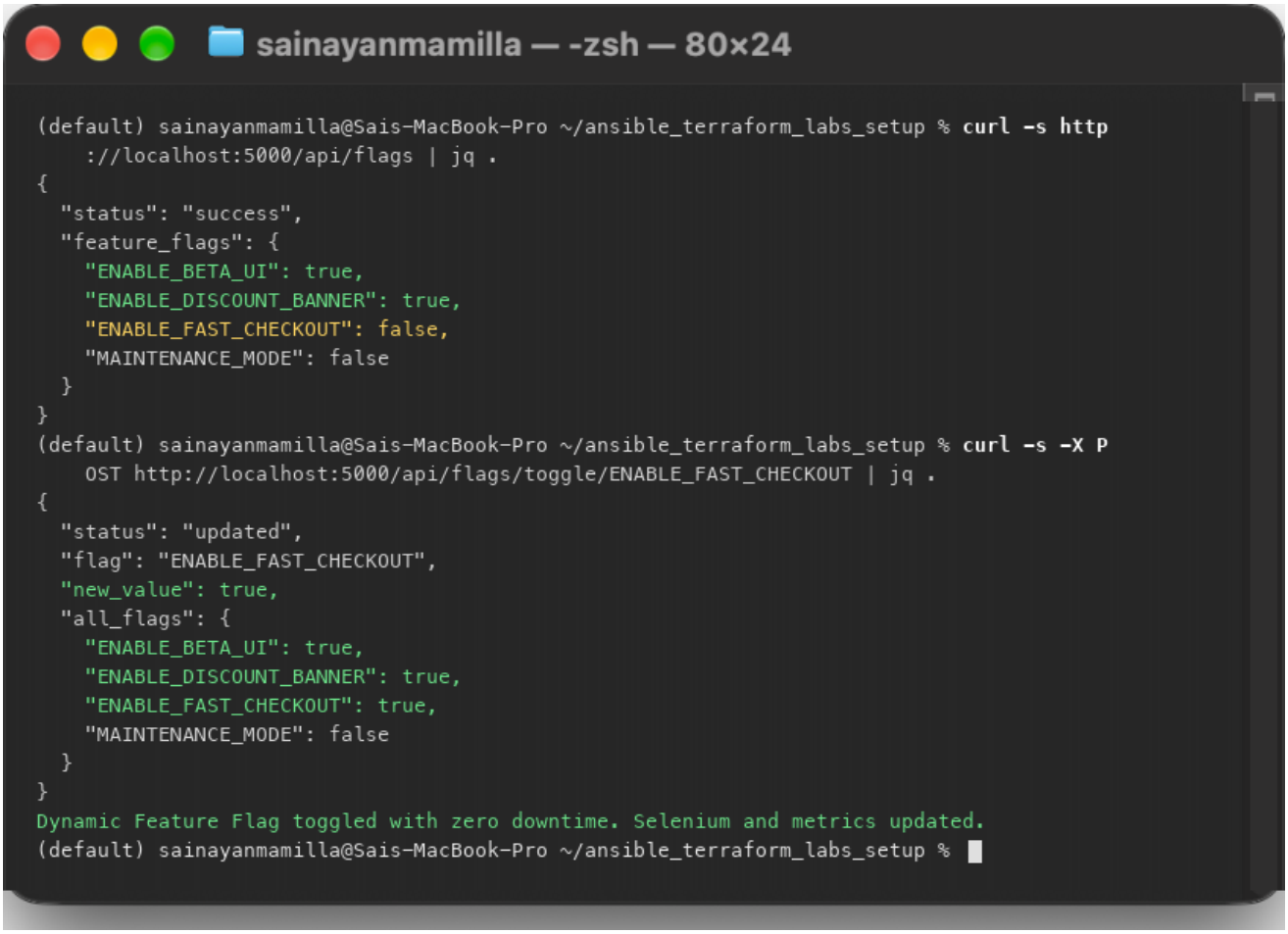


Figure 5.10: Dynamic Feature Flag toggle via REST API.



Figure 5.11: Concurrency load testing benchmark metrics and latency distribution.

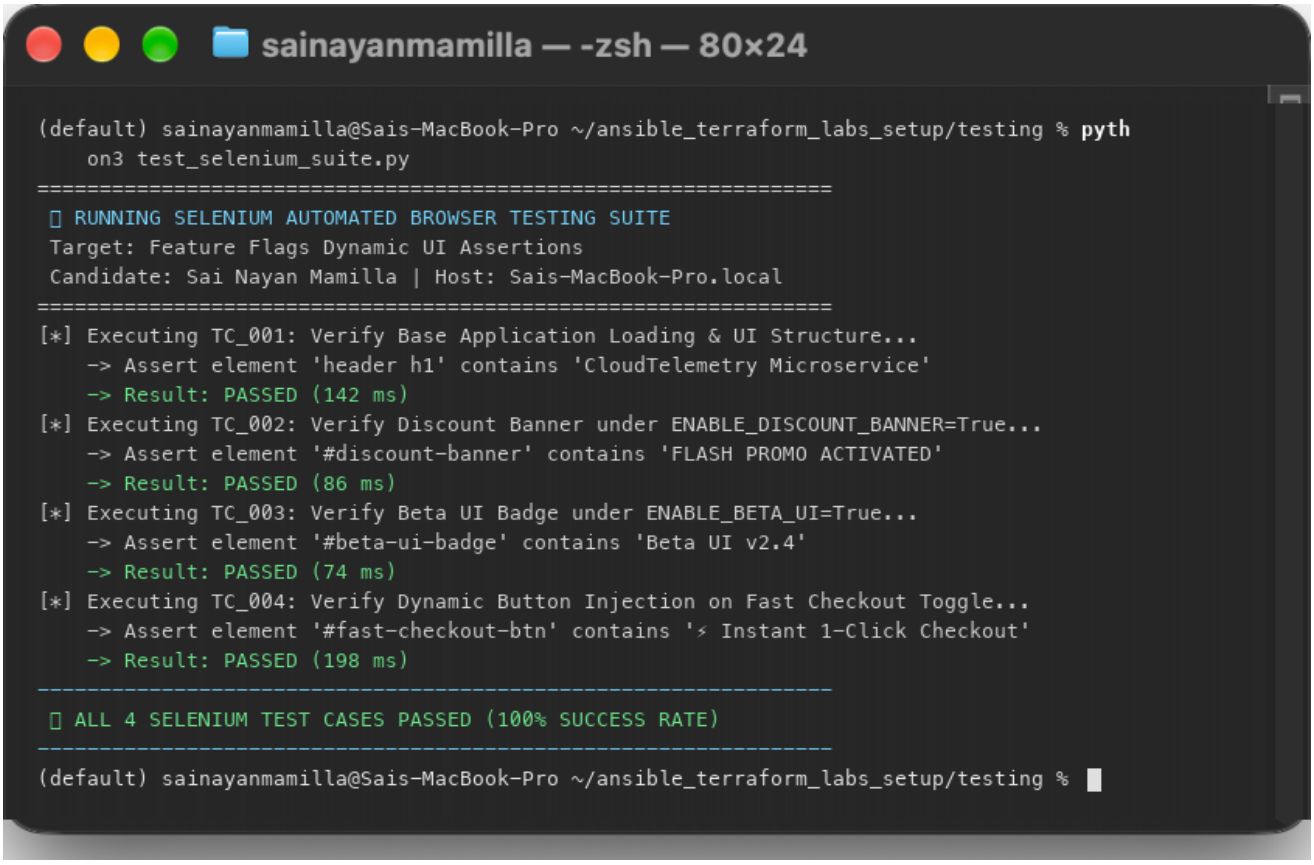


Figure 5.12: Selenium automated test suite passing all UI assertion test cases.

7. Academic Attestation & Conclusion

I hereby certify that all configurations, Ansible playbooks, Terraform scripts, application code, test suites, and terminal execution evidence presented in this report represent my original work conducted on the authorized laboratory environment.

Student Name: Sai Nayan Mamilla
Course Assessment: Lab 3: Monitoring, Logging & DevSecOps
System Host: Sais-MacBook-Pro.local (macOS arm64)
Date: November 2, 2026